CIS 3300 · CLASSES AND OBJECTS REVIEW, PART 3 OF 3

Object Serialization

Saving whole objects to bytes, restoring them, and why to be careful

---

In Week 7 you saved data as JSON and worked with raw bytes. Both times, you decided field by field what to write and how to read it back. Java also has a built-in mechanism, called *object serialization*, that saves an entire object, and every object it refers to, in one call. This set of notes shows how it works using the `Student` record and `LetterGrade` enum from part 2, what does and does not get saved, and then the most important part: why serialization is risky, and why JSON is usually the better choice for data exchange. For reading, see Core Java, Volume II, Chapter 2, Section 2.3.

A note before you begin. These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

1. What Serialization Is

Serialization turns an object into a sequence of bytes that can be written to a file or sent over a network. *Deserialization* reads those bytes and rebuilds the object. Two stream classes from `java.io` do the work, and they wrap the byte streams you already know:

- `ObjectOutputStream`, whose `writeObject` method serializes an object.
- `ObjectInputStream`, whose `readObject` method deserializes one.

A class allows its objects to be serialized by implementing `java.io.Serializable`. That interface has no methods at all. It is a *marker interface*: implementing it simply tells Java that you agree to let objects of this class be saved this way.

2. Making the Gradebook Serializable

The `Student` record from part 2 needs one change, adding `Serializable` to its `implements` list:

```
import java.io.Serializable;

public record Student(String id, String name, double percent)
        implements Comparable<Student>, Serializable {
    // The body is unchanged from part 2.
}
```

Enums are always serializable, so `LetterGrade` needs no change. Next, a `Gradebook` class holds a course code and a list of students.

```
import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

public class Gradebook implements Serializable {
    private static final long serialVersionUID = 1L;

    private final String courseCode;
    private final ArrayList<Student> students = new ArrayList<>();

    // Derived from the student list, so there is no reason to save it.
    private transient Double cachedAverage;   // null means "not computed yet"

    public Gradebook(String courseCode) {
        this.courseCode = courseCode;
    }

    public void add(Student s) {
        students.add(s);
        cachedAverage = null;   // the old average is now stale
    }

    public double average() {
        if (cachedAverage == null) {
            double sum = 0.0;
            for (Student s : students) {
                sum += s.percent();
            }
            cachedAverage = students.isEmpty() ? 0.0 : sum / students.size();
        }
        return cachedAverage;
    }

    public String getCourseCode() { return courseCode; }
    public List<Student> getStudents() { return List.copyOf(students); }

    // For the demo only: shows whether the cache survived.
    public boolean isAverageCached() { return cachedAverage != null; }
}
```

Two new things appear here, `serialVersionUID` and `transient`. Both are explained below.

3. Saving and Loading

Saving takes one call to `writeObject`.

```
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;
```

```
public class SaveGradebook {
    public static void main(String[] args) throws IOException {
        Gradebook book = new Gradebook("CIS 3300");
        book.add(Student.create("Ada", 91.5));
        book.add(Student.create("Grace", 84.0));
        book.add(Student.create("Alan", 72.5));
        System.out.println("Average: " + book.average());
        System.out.println("Cached before saving? " + book.isAverageCached());

        try (ObjectOutputStream out =
                 new ObjectOutputStream(new FileOutputStream("gradebook.ser"))) {
            out.writeObject(book);
        }
        System.out.println("Saved.");
    }
}
```

```
Average: 82.66666666666667
Cached before saving? true
Saved.
```

Loading takes one call to `readObject`. Run this as a separate program, so it starts in a fresh JVM the way a real program would the next day.

```
import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

public class LoadGradebook {
    public static void main(String[] args)
            throws IOException, ClassNotFoundException {
        Gradebook book;
        try (ObjectInputStream in =
                 new ObjectInputStream(new FileInputStream("gradebook.ser"))) {
            book = (Gradebook) in.readObject();   // readObject returns Object
        }
        System.out.println(book.getCourseCode());
        for (Student s : book.getStudents()) {
            System.out.println(s);
        }
        System.out.println("Cached after loading? " + book.isAverageCached());
        System.out.println("Average: " + book.average());

        Student newcomer = Student.create("Barbara", 88.0);
        System.out.println("New student ID: " + newcomer.id());
    }
}
```

```
CIS 3300
Student[id=S1, name=Ada, percent=91.5]
Student[id=S2, name=Grace, percent=84.0]
```

```
Student[id=S3, name=Alan, percent=72.5]
Cached after loading? false
Average: 82.66666666666667
New student ID: S1
```

A few details about `readObject`:

- It returns `Object`, so you must cast the result to the type you expect.
- It can throw `ClassNotFoundException`, a checked exception, if the bytes name a class that the running program does not have.
- Objects must be read back in the same order they were written.

Look closely at the last two lines of that output. The next section explains both.

4. What Gets Saved, and What Does Not

The whole object graph

`writeObject` saves the object's instance fields, and then follows every reference and saves those objects too. Saving the `Gradebook` saved its `ArrayList`, every `Student` in the list, and every `String` inside each student. Every object reached this way must be serializable. If one is not, the write fails with an exception that names the offending class:

```
java.io.NotSerializableException: Student
```

Shared references are preserved

If the same object is referenced twice, it is written once, and after reading, both references point to the same rebuilt object. This program saves a list that contains one student twice, plus an enum constant.

```
Student ada = Student.create("Ada", 91.5);
ArrayList<Object> items = new ArrayList<>();
items.add(ada);
items.add(ada);              // the same object, twice
items.add(LetterGrade.B);
// ... write items to a stream, then read it back into copy ...
System.out.println(copy.get(0) == copy.get(1));   // true:  still one shared object
System.out.println(copy.get(0) == ada);           // false: a new object was built
System.out.println(copy.get(2) == LetterGrade.B); // true:  the same enum constant
```

The middle line is worth remembering. Deserialization always builds *new* objects. The copy is equal to the original, but it is not the original. Enums are the exception, as we will see shortly.

Static fields are not saved

Static fields belong to the class, not to any object, so they are never written. That explains the surprise at the end of the loading program. The new program started with `nextId` back at 1, so

the next student was given `S1`, an ID that Ada already has. The saved data and the static counter are now out of step. Anything static that must survive between runs has to be saved separately or rebuilt from the loaded data.

Transient fields are not saved

A field marked `transient` is skipped when writing. When the object is read back, that field holds its default value: `null` for references, `0` for numbers, and `false` for booleans. That is why `isAverageCached()` printed `false` after loading. Because `average()` recomputes the value when the cache is `null`, the program still gets the right answer.

Good candidates for `transient`:

- Cached or derived values that can be recomputed, like `cachedAverage`.
- Sensitive data that should never be written to a file, such as a password.
- Things that make no sense outside the running program, such as an open file or a database connection, which cannot be serialized anyway.

Table 1: What serialization saves.

Kind of data	Saved?	After loading
Ordinary instance fields	Yes	Restored with their saved values
Referenced objects	Yes	Rebuilt as new objects; shared references stay shared
`transient` **fields**	No	Default value: `null`, `0`, or `false`
`static` **fields**	No	Whatever the class currently has in the running program

5. The serialVersionUID

A serialized file can outlive the version of the class that wrote it. Every serializable class has a version number, `serialVersionUID`, that is written into the stream. When reading, Java compares the number in the file with the number in the current class. If they differ, the read fails:

```
java.io.InvalidClassException: Gradebook; local class incompatible:
  stream classdesc serialVersionUID = 1, local class serialVersionUID = 2
```

If you do not declare the number, Java computes one from details of the class, such as its fields and methods. Even a harmless change, like adding a method, can change that computed number and make every old file unreadable. Declaring it yourself, as `Gradebook` does, puts you in control: keep the number the same when a change is compatible, and change it on purpose when old files should be rejected.

6. Records and Enums Are Special

Enums are written by name only. Reading one back does not build a new object; it returns the existing constant with that name. That is why `copy.get(2) == LetterGrade.B` was `true`. The

enum's fields are not written either, so changing `gradePoints` later does not affect saved data. Renaming a constant, however, means old files that contain the old name can no longer be read.

Records are written as their components. When a record is read back, Java rebuilds it by calling its canonical constructor, so the compact constructor runs and validates the data.

Ordinary classes are different, and this is the key point for the next section. When an object of an ordinary serializable class is read back, its constructor does *not* run. Java fills in the fields directly from the bytes. Any checks you wrote in that constructor are skipped.

For records and enums, Java ignores `serialVersionUID`, so you do not need to declare one.

7. The Security Risk of Deserialization

Deserialization builds objects from bytes, and whoever wrote those bytes decides what comes out. If the bytes come from a source you do not control, such as a file a user uploads or data that arrives over a network, you are letting a stranger construct objects inside your program.

Seeing it happen

This program creates two objects that both reject a percent above 100: an ordinary class and a record. It serializes each one, edits the bytes to change the stored 85.0 to 150.0, the way an attacker could edit a file, and then deserializes the result.

```
import java.io.*;
import java.nio.ByteBuffer;

public class TamperDemo {
    // An ordinary class that checks its data in the constructor.
    static class ScoreClass implements Serializable {
        private static final long serialVersionUID = 1L;
        private final double percent;
        ScoreClass(double percent) {
            if (percent < 0 || percent > 100) {
                throw new IllegalArgumentException("bad percent: " + percent);
            }
            this.percent = percent;
        }
        double percent() { return percent; }
    }

    // A record with the same check in its compact constructor.
    record ScoreRecord(double percent) implements Serializable {
        ScoreRecord {
            if (percent < 0 || percent > 100) {
                throw new IllegalArgumentException("bad percent: " + percent);
            }
        }
    }

    public static void main(String[] args) throws Exception {
        Object fromClass = tamper(new ScoreClass(85.0));
        System.out.println("Class came back with " + ((ScoreClass) fromClass).percent()
```

```
    );

        try {
            tamper(new ScoreRecord(85.0));
        } catch (InvalidObjectException e) {
            System.out.println("Record rejected: " + e.getMessage());
        }
    }

    // Serialize, overwrite the stored 85.0 with 150.0, then deserialize.
    static Object tamper(Object original) throws Exception {
        ByteArrayOutputStream buffer = new ByteArrayOutputStream();
        try (ObjectOutputStream out = new ObjectOutputStream(buffer)) {
            out.writeObject(original);
        }
        byte[] bytes = buffer.toByteArray();
        byte[] good = ByteBuffer.allocate(8).putDouble(85.0).array();
        byte[] evil = ByteBuffer.allocate(8).putDouble(150.0).array();
        for (int i = 0; i + 8 <= bytes.length; i++) {
            if (java.util.Arrays.equals(bytes, i, i + 8, good, 0, 8)) {
                System.arraycopy(evil, 0, bytes, i, 8);
                break;
            }
        }
        try (ObjectInputStream in = new ObjectInputStream(new ByteArrayInputStream(
    bytes))) {
            return in.readObject();
        }
    }
}
```

```
Class came back with 150.0
Record rejected: bad percent: 150.0
```

The ordinary class came back holding a value its own constructor says is impossible. The record refused, because its compact constructor ran during deserialization. That is one more reason to prefer records for plain data.

It gets worse than bad values

A bad percent is the mild version. `readObject` can build an object of *any* serializable class available to your program, including classes from libraries you included, and some classes run code of their own while they are being deserialized. Attackers have learned to chain such classes together so that simply reading a crafted stream runs commands they choose. The cast to `Gradebook` does not help, because the damage is done inside `readObject`, before the cast ever happens.

This is not a theoretical risk. In January 2015, security researchers Chris Frohoff and Gabriel Lawrence showed that a crafted stream could use classes from a popular library, Apache Commons Collections, to run commands of the attacker's choosing [1]. Later that year, Stephen Breen of FoxGlove Security used their technique against widely used Java servers, including WebLogic, WebSphere, JBoss, and Jenkins [2], and the Apache Commons team published a response [3]. "Insecure

deserialization" was ranked eighth on the 2017 OWASP Top Ten list of web application security risks [4]. In the 2021 list, it is part of the broader category "Software and Data Integrity Failures" [5].

Java's own experts reached the same conclusion. In *Effective Java*, Item 85, "Prefer alternatives to Java serialization," Joshua Bloch writes: "There is no reason to use Java serialization in any new system you write" [6].

If you must deserialize

- Never deserialize data from a source you do not trust.
- Install an `ObjectInputFilter` that allows only the classes you expect and rejects everything else.

```
try (ObjectInputStream in =
         new ObjectInputStream(new FileInputStream(fileName))) {
    // Allow only the classes we expect; reject everything else.
    in.setObjectInputFilter(ObjectInputFilter.Config.createFilter(
        "maxdepth=10;Gradebook;Student;java.util.ArrayList;java.lang.Object;!*"));
    return (Gradebook) in.readObject();
}
```

The pattern is a list separated by semicolons. `maxdepth=10` limits how deeply objects can nest. Each class name is allowed. `java.lang.Object` is needed because an `ArrayList` stores its elements in an `Object` array internally. The final `!*` rejects every other class, and a stream that contains one fails with `java.io.InvalidClassException: filter status: REJECTED`.

A filter blocks unexpected *classes*. It does not check *values*, so the tampered `ScoreClass` above would still get through. A filter reduces the risk; it does not remove it.

Sources for this section

[1] C. Frohoff and G. Lawrence, "Marshalling Pickles," AppSecCali, January 2015. Their tool, ysoserial: `https://github.com/frohoff/ysoserial`

[2] S. Breen, "What Do WebLogic, WebSphere, JBoss, Jenkins, OpenNMS, and Your Application Have in Common? This Vulnerability," FoxGlove Security, November 6, 2015. `https://foxglovesecurity.com/2015/11/06/what-do-weblogic-websphere-jboss-jenkins-opennms-and-your-application-have-in-common-this-vulnerability/`

[3] B. Eckenfels and G. Gregory, "Apache Commons statement to widespread Java object deserialisation vulnerability," The Apache Software Foundation Blog, November 10, 2015. `https://blogsarchive.apache.org/foundation/entry/apache_commons_statement_to_widespread`

[4] OWASP, "A8:2017 Insecure Deserialization," OWASP Top Ten 2017. `https://owasp.org/www-project-top-ten/2017/A8_2017-Insecure_Deserialization`

[5] OWASP, "A08:2021 Software and Data Integrity Failures," OWASP Top Ten 2021. `https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures/`

[6] J. Bloch, *Effective Java*, 3rd ed., Addison-Wesley, 2018. Chapter 12, Item 85, "Prefer alternatives to Java serialization."

8. Serialization or JSON?

Here is the same gradebook as JSON, the format you used in Week 7:

```
{
  "courseCode": "CIS 3300",
  "students": [
    { "id": "S1", "name": "Ada",   "percent": 91.5 },
    { "id": "S2", "name": "Grace", "percent": 84.0 },
    { "id": "S3", "name": "Alan",  "percent": 72.5 }
  ]
}
```

When you read this back, your own code calls `new Student(...)` for each entry, so the compact constructor checks every value, and the file can only ever produce the kinds of objects your code chooses to build.

Table 2: Java serialization compared with JSON.

Question	Java serialization	JSON
Who can read it?	Only Java programs that have the same classes	Almost any language or tool
Can a person read it?	No; it is binary	Yes; it is text
How much code?	Very little; one call each way	More; you map fields yourself
What is restored?	The full object graph, including shared references	Plain data; your code rebuilds the objects
Are constructors run?	Only for records	Yes, because your code calls them
Safe with untrusted input?	No; risky even with a filter	Much safer, since only data is read
After changing a class?	Old files can break	You control how old data is read

The bottom line. Use JSON, or another plain data format, for anything that leaves your program: files other people or programs will read, data sent over a network, and anything that must survive changes to your classes. Java serialization is reasonable only for data your own program wrote, reads back, and fully trusts. You will still meet it in older code and in some libraries, which is why it is worth understanding.

9. Summary

Table 3: Summary of the ideas in these notes.

Idea	Key point
`Serializable`	A marker interface that allows a class's objects to be serialized
Object graph	Writing one object writes everything it refers to; all of it must be serializable
`transient`	Skips a field; it comes back as `null`, `0`, or `false`
Static fields	Never saved; they belong to the class, not the object
`serialVersionUID`	A version stamp; declare it so harmless changes do not break old files
Records and enums	Records are rebuilt through their constructor; enums are restored by name
Security	Deserializing untrusted bytes is dangerous; prefer JSON for data exchange

10. Try It

1. Remove `Serializable` from the `Student` record and run `SaveGradebook`. Which exception do you get, and which class does it name? Why that class and not `Gradebook`?
2. Remove `transient` from `cachedAverage`, then save and load again. What is printed by `isAverageCached()` now, and why does the program still work?
3. Fix the duplicate ID problem. Add a static method to `Student` that moves `nextId` past a given number, and call it after loading with the highest ID found in the gradebook.